

# TP6 — Transformer un modèle YANG en configuration exploitable

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

## Description

YANG est un langage de modélisation utilisé (avec NETCONF/RESTCONF) pour décrire précisément la structure et les contraintes des données de configuration réseau — c’est ce que Cisco et la plupart des constructeurs utilisent pour définir “à quoi doit ressembler” une config valide. Ce TP écrit un modèle YANG simplifié pour les interfaces réseau NexaCorp, puis transforme une instance de données conforme à ce modèle en variables Ansible (host\_vars) exploitables — la brique qui alimentera les templates Jinja2 du TP7.

## Prérequis

- TP1 à TP5 validés
- Python et pip accessibles

## Étapes

### Étape 1 — Créer sa branche pour ce TP

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour2-tp6
```

### Étape 2 — Installer pyang (outil de validation YANG)

```
pip install pyang
pyang --version
```

### Étape 3 — Écrire le modèle YANG

```
mkdir -p yang
cat > yang/nexacorp-interfaces.yang << 'EOF'
module nexacorp-interfaces {
    yang-version 1.1;
```

```

namespace "urn:nexacorp:interfaces";
prefix "nxc-if";

organization "NexaCorp";
description "Modele YANG simplifie des interfaces reseau NexaCorp.";

typedef admin-state {
    type enumeration {
        enum up;
        enum down;
    }
    description "Etat administratif souhaite pour l'interface.";
}

list interface {
    key "name";
    description "Une interface reseau geree sur un site NexaCorp.";

    leaf name {
        type string;
        description "Nom de l'interface, ex: GigabitEthernet0/1.";
    }

    leaf description {
        type string;
        description "Description libre de l'interface.";
    }

    leaf enabled {
        type admin-state;
        default up;
        description "Etat administratif souhaite : up ou down.";
    }

    leaf vlan-id {
        type uint16 {
            range "1..4094";
        }
        mandatory true;
        description "Identifiant VLAN associe a l'interface.";
    }
}
}
EOF

```

## Étape 4 — Valider le modèle

```
pyang --strict yang/nexacorp-interfaces.yang
```

Aucune sortie = modèle valide. Toute erreur de syntaxe YANG s'afficherait ici.

## Étape 5 — Visualiser la structure du modèle

```
pyang -f tree yang/nexacorp-interfaces.yang
```

### Résultat attendu :

```
module: nexacorp-interfaces
  +--rw interface* [name]
    +--rw name          string
    +--rw description?  string
    +--rw enabled?      admin-state
    +--rw vlan-id       uint16
```

\* = liste (plusieurs interfaces), ? = optionnel, rien = obligatoire. C'est cette structure qu'on va respecter dans les données réelles.

```
pyang -f sample-xml-skeleton yang/nexacorp-interfaces.yang
```

Génère un squelette XML vide conforme au modèle — utile pour voir “à quoi ressemblerait” une vraie donnée NETCONF.

## Étape 6 — Écrire une instance de données conforme au modèle

```
cat > inventory/network_model.yaml << 'EOF'
sites:
  siege:
    interfaces:
      - name: GigabitEthernet0/1
        description: "Lien vers coeur de reseau"
        enabled: up
        vlan-id: 10
      - name: GigabitEthernet0/2
        description: "Reserve"
        enabled: down
        vlan-id: 20
  agence1:
    interfaces:
      - name: GigabitEthernet0/1
        description: "LAN utilisateurs agence1"
        enabled: up
        vlan-id: 30
      - name: GigabitEthernet0/2
        description: "Reserve"
        enabled: down
        vlan-id: 20
  agence2:
    interfaces:
      - name: GigabitEthernet0/1
```

```

        description: "LAN utilisateurs agence2"
        enabled: up
        vlan-id: 40
-   name: GigabitEthernet0/2
    description: "Reserve"
    enabled: down
    vlan-id: 20

```

EOF

## Étape 7 — Écrire le script de transformation (YANG → host\_vars Ansible)

```

cat > scripts/yang_to_config.py << 'EOF'
"""

```

Transforme une instance de donnees (inventory/network\_model.yaml), conforme a la structure definie par yang/nexacorp-interfaces.yang, en fichiers host\_vars Ansible exploitables (un fichier par site).

```

"""

```

```

import os
import sys
import yaml

```

```

HERE = os.path.dirname(os.path.abspath(__file__))
MODEL_FILE = os.path.join(HERE, "..", "inventory", "network_model.yaml")
HOST_VARS_DIR = os.path.join(HERE, "..", "inventory", "host_vars")

```

```

ALLOWED_STATES = {"up", "down"}

```

```

class YangValidationError(Exception):
    """Erreur levee quand une instance ne respecte pas le modele YANG."""
    pass

```

```

def validate_interface(site, iface):
    if "name" not in iface or not isinstance(iface["name"], str):
        raise YangValidationError(f"[{site}] 'name' manquant ou invalide : {iface}")

    if "vlan-id" not in iface:
        raise YangValidationError(f"[{site}] 'vlan-id' obligatoire manquant pour {iface.get('name')}")
    vlan_id = iface["vlan-id"]
    if not isinstance(vlan_id, int) or not (1 <= vlan_id <= 4094):
        raise YangValidationError(f"[{site}] vlan-id hors plage (1-4094) pour {iface.get('name')} : {vlan_id}")

    enabled = iface.get("enabled", "up")
    if enabled not in ALLOWED_STATES:
        raise YangValidationError(f"[{site}] 'enabled' doit etre 'up' ou 'down' pour {iface.get('name')} : {enabled}")

```

```

def main():
    with open(MODEL_FILE) as f:
        data = yaml.safe_load(f)

    os.makedirs(HOST_VARS_DIR, exist_ok=True)

    for site, site_data in data["sites"].items():
        interfaces = site_data["interfaces"]

        for iface in interfaces:
            validate_interface(site, iface)

        host_vars = {"interfaces": interfaces}
        out_path = os.path.join(HOST_VARS_DIR, f"{site}.yaml")
        with open(out_path, "w") as f:
            f.write("---\n")
            yaml.dump(host_vars, f, default_flow_style=False, sort_keys=False)

        print(f"OK : {len(interfaces)} interface(s) validees et ecrites dans {out_path}")

if __name__ == "__main__":
    try:
        main()
    except YangValidationError as e:
        print(f"ERREUR de validation YANG : {e}", file=sys.stderr)
        sys.exit(1)
EOF

```

## Étape 8 — Exécuter la transformation

```

python scripts/yang_to_config.py
cat inventory/host_vars/siege.yaml

```

### Résultat attendu :

```

OK : 2 interface(s) validees et ecrites dans ../inventory/host_vars/siege.yaml
OK : 2 interface(s) validees et ecrites dans ../inventory/host_vars/agence1.yaml
OK : 2 interface(s) validees et ecrites dans ../inventory/host_vars/agence2.yaml

```

## Étape 9 — Tester le rejet d'une donnée invalide

```

sed -i 's/vlan-id: 10/vlan-id: 9999/' inventory/network_model.yaml
python scripts/yang_to_config.py

```

Doit afficher une erreur claire (vlan-id hors plage (1-4094)) et s'arrêter (code de sortie 1), sans écrire de fichier incorrect. Remettez la valeur correcte ensuite :

```

sed -i 's/vlan-id: 9999/vlan-id: 10/' inventory/network_model.yaml
python scripts/yang_to_config.py

```

## Étape 10 — Committer et pousser

```
git add yang/ scripts/yang_to_config.py inventory/network_model.yaml inventory/host_vars/  
git commit -m "TP6 : modele YANG et transformation en host_vars Ansible"  
git push -u origin <prenom>-<nom>-jour2-tp6
```

Ouvrir une pull request <prenom>-<nom>-jour2-tp6 → dev.

## Comprendre le TP

Le modèle YANG (`nexacorp-interfaces.yang`) ne contient aucune donnée — il décrit uniquement la forme que doivent avoir les données : une liste d’interfaces, chacune avec un nom (obligatoire), une description (optionnelle), un état (up/down uniquement), et un VLAN (obligatoire, entre 1 et 4094). C’est un contrat, pas un contenu.

`inventory/network_model.yaml` est une instance de données qui respecte ce contrat — les vraies valeurs pour les 3 sites NexaCorp. `yang_to_config.py` joue le rôle qu’un vrai validateur NETCONF jouerait automatiquement : vérifier que chaque interface respecte les règles du modèle (VLAN dans la bonne plage, état autorisé, champs obligatoires présents) avant de produire une configuration exploitable. Si une donnée ne respecte pas le contrat, elle est rejetée avant même d’atteindre Ansible — exactement le genre d’erreur qui, dans le scénario NexaCorp du début (le déplacement de 2 jours puis la nuit sur place), aurait dû être attrapée avant le déploiement plutôt qu’après.

Le résultat (`inventory/host_vars/<site>.yaml`) est directement exploitable par Ansible : au TP7, les templates Jinja2 liront ces mêmes fichiers `host_vars` pour générer les configurations finales des équipements.

## Validation

- ☐ `pyang --strict yang/nexacorp-interfaces.yang` valide sans erreur
- ☐ `pyang -f tree` affiche la structure attendue (liste interface\* avec les 4 leafs)
- ☐ `python scripts/yang_to_config.py` génère un fichier `host_vars/<site>.yaml` par site
- ☐ Le contenu de `host_vars/siege.yaml` correspond aux interfaces définies dans `network_model.yaml`
- ☐ Une valeur `vlan-id` hors plage (ex. 9999) fait échouer le script avec un message clair, sans écrire de fichier
- ☐ Une pull request <prenom>-<nom>-jour2-tp6 → dev est ouverte